

R Programming Tutorial



R Programming Tutorial is designed for both beginners and professionals. Our tutorial provides all the basic and advanced concepts of data analysis and visualization.

R is a software environment which is used to analyze statistical information and graphical representation. R allows us to do modular programming using functions.

Our R tutorial includes all topics of R such as introduction, features, installation, rstudio ide, variables, datatypes, operators, if statement, vector, data handing, graphics, statistical modelling, etc. This programming language was named R, based on the first name letter of the two authors (Robert Gentleman and Ross Ihaka).

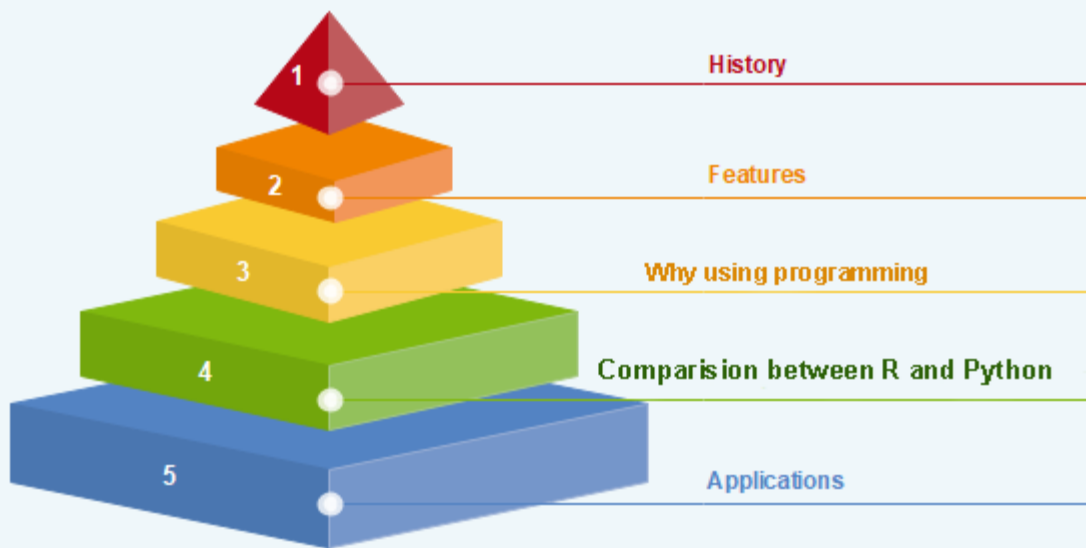
What is R Programming

"R is an interpreted computer programming language which was created by Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand." The **R Development Core Team** currently develops R. It is also a software environment used to analyze **statistical information, graphical representation, reporting, and data modeling**. R is the implementation of the **S programming** language, which is combined with **lexical scoping semantics**.

R not only allows us to do branching and looping but also allows to do modular programming using functions. R allows integration with the procedures written in the C, C++, .Net, Python, and FORTRAN languages to improve efficiency.

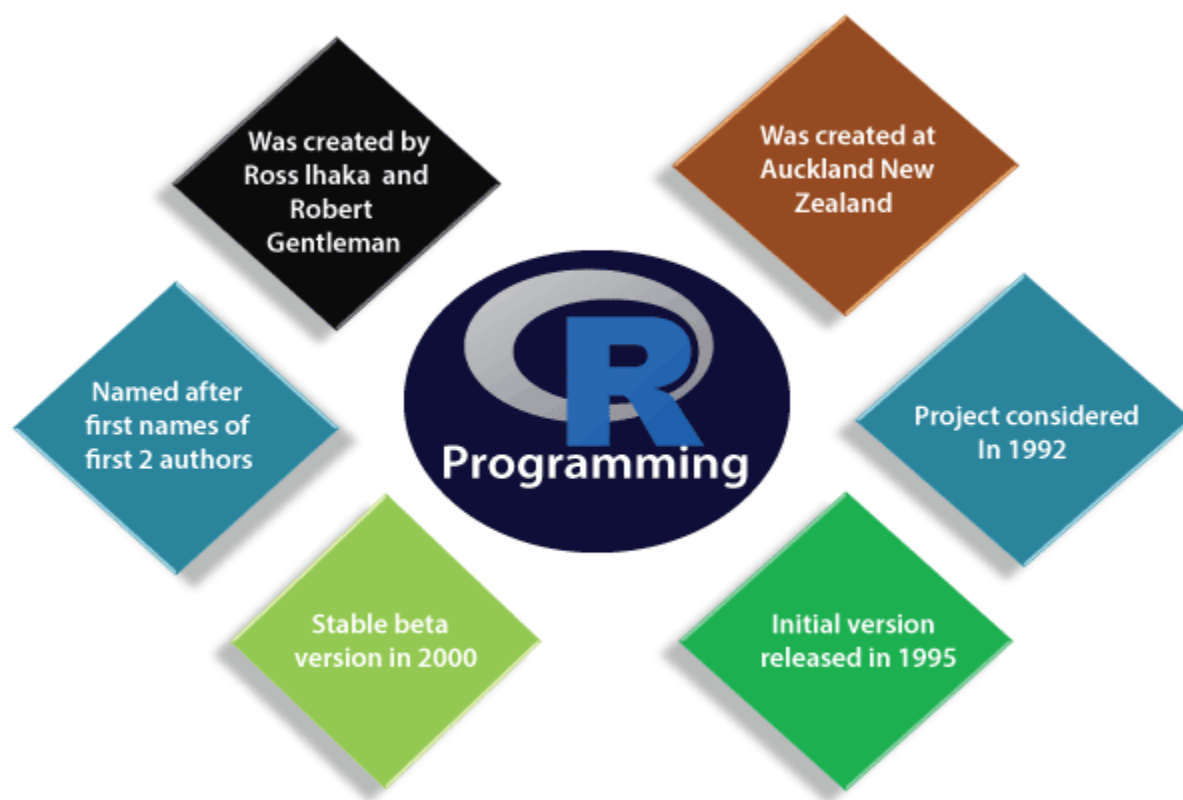
In the present era, R is one of the most important tool which is used by researchers, data analyst, statisticians, and marketers for retrieving, cleaning, analyzing, visualizing, and presenting data.

R Programming



History of R Programming

The history of R goes back about 20-30 years ago. R was developed by Ross Ihaka and Robert Gentleman in the University of Auckland, New Zealand, and the R Development Core Team currently develops it. This programming language name is taken from the name of both the developers. The first project was considered in 1992. The initial version was released in 1995, and in 2000, a stable beta version was released.



The following table shows the release date, version, and description of R language:

Version-Release	Date	Description
0.49	1997-04-23	First time R's source was released, and CRAN (Comprehensive R Archive Network) was started.
0.60	1997-12-05	R officially gets the GNU license.
0.65.1	1999-10-07	update.packages and install.packages both are included.
1.0	2000-02-29	The first production-ready version was released.
1.4	2001-12-19	First version for Mac OS is made available.
2.0	2004-10-04	The first version for Mac OS is made available.
2.1	2005-04-18	Add support for UTF-8encoding, internationalization, localization etc.
2.11	2010-04-22	Add support for Windows 64-bit systems.
2.13	2011-04-14	Added a function that rapidly converts code to byte code.

2.14	2011-10-31	Added some new packages.
2.15	2012-03-30	Improved serialization speed for long vectors.
3.0	2013-04-03	Support for larger numeric values on 64-bit systems.
3.4	2017-04-21	The just-in-time compilation (JIT) is enabled by default.
3.5	2018-04-23	Added new features such as compact internal representation of integer sequences, serialization format etc.

Features of R programming

R is a domain-specific programming language which aims to do data analysis. It has some unique features which make it very powerful. The most important arguably being the notation of vectors. These vectors allow us to perform a complex operation on a set of values in a single command. There are the following features of R programming:

1. It is a simple and effective programming language which has been well developed.
2. It is data analysis software.
3. It is a well-designed, easy, and effective language which has the concepts of user-defined, looping, conditional, and various I/O facilities.
4. It has a consistent and incorporated set of tools which are used for data analysis.
5. For different types of calculation on arrays, lists and vectors, R contains a suite of operators.
6. It provides effective data handling and storage facility.
7. It is an open-source, powerful, and highly extensible software.
8. It provides highly extensible graphical techniques.
9. It allows us to perform multiple calculations using vectors.
10. R is an interpreted language.

Why use R Programming?

There are several tools available in the market to perform data analysis. Learning new languages is time taken. The data scientist can use two excellent tools, i.e., R and Python. We may not have time to learn them both at the time when we get started to learn data science. Learning statistical modeling and algorithm is more important than to learn a programming language. A programming language is used to compute and communicate our discovery.

The important task in data science is the way we deal with the data: clean, feature engineering, feature selection, and import. It should be our primary focus. Data scientist job is to understand the data, manipulate it, and expose the best approach. For machine learning, the best algorithms can be implemented with R. **Keras** and **TensorFlow** allow us to create high-end machine learning techniques. R has a package to perform **Xgboost**. Xgboost is one of the best algorithms for **Kaggle competition**.

R communicate with the other languages and possibly calls Python, Java, C++. The big data world is also accessible to R. We can connect R with different databases like **Spark** or **Hadoop**.

In brief, R is a great tool to investigate and explore the data. The elaborate analysis such as clustering, correlation, and data reduction are done with R.

Applications of R

There are several-applications available in real-time. Some of the popular applications are as follows:

- Facebook
- Google
- Twitter
- HRDAG
- Sunlight Foundation
- RealClimate
- NDAA
- XBOX ONE
- ANZ
- FDA

How to Download R and R Studio?

To Install R and R Studio on Windows we will have to download R and R Studio with the following steps.

Step 1: First, you need to set up an R environment in your local machine. You can download the same from r-project.org.

Don't want to download or install anything? Get started with RStudio on [Posit Cloud for free](#). If you're a professional data scientist looking to download RStudio and also need common enterprise features, don't hesitate to [book a call with us](#).

1: Install R

RStudio requires R 3.3.0+. Choose a version of R that matches your computer's operating system.

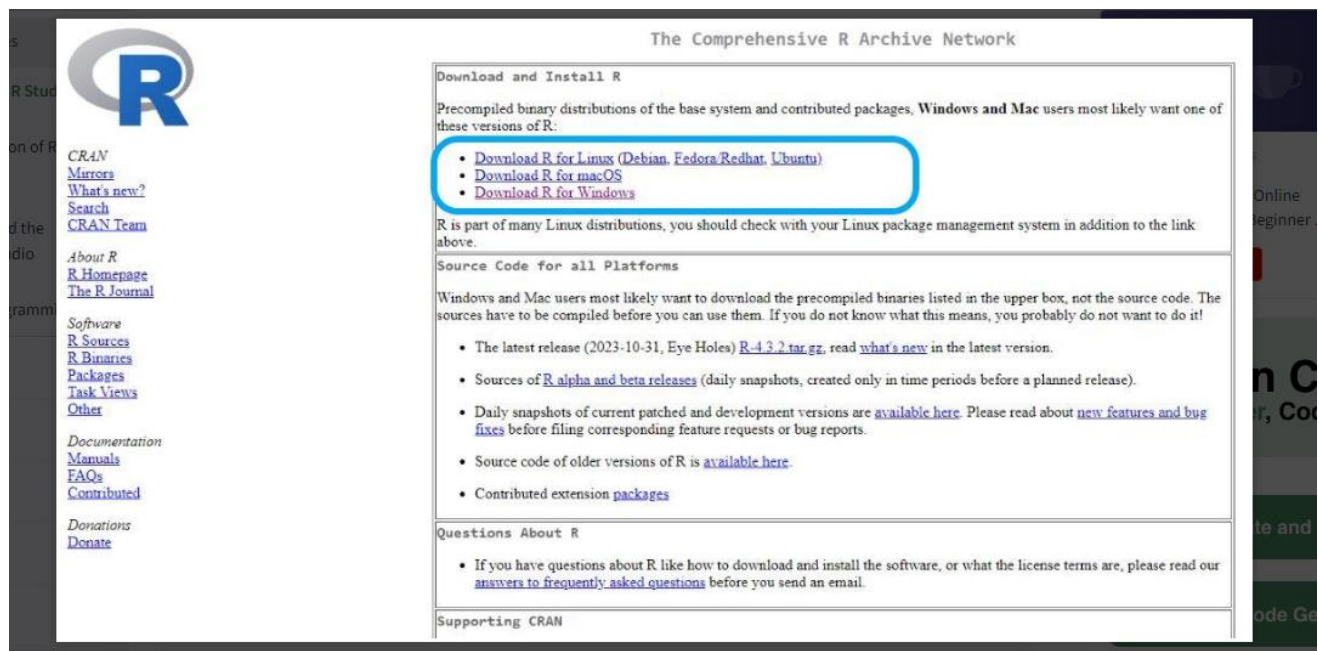
DOWNLOAD AND INSTALL R

2: Install RStudio

DOWNLOAD RSTUDIO DESKTOP FOR WINDOWS

Size: 214.34 MB | [SHA-256: FE62B784](#) | Version: 2023.09.1+494 | Released: 2023-10-17

Install R and R Studio



The screenshot shows the CRAN website with the R logo on the left. The main content area is titled "The Comprehensive R Archive Network" and "Download and Install R". It provides instructions for downloading precompiled binary distributions for Windows and Mac users. A blue box highlights the download links for Linux (Debian, Fedora, Redhat, Ubuntu), macOS, and Windows. Below this, it mentions that R is part of many Linux distributions and suggests checking with the Linux package management system. It also provides links for source code for all platforms, including the latest release (2023-10-31, Eye Holes) R-4.3.2 tar.gz, sources of R alpha and beta releases, daily snapshots of current patched and development versions, and source code of older versions of R. The page also includes a section for "Questions About R" and a "Supporting CRAN" section.

here we can select the linux,mac or windows any one according to users system. you have to click on for which you want to install.

The screenshot shows the CRAN website's download page for R-4.3.2 for Windows. The R logo is in the top left. A sidebar on the left contains links: CRAN, Mirrors, What's new?, Search, CRAN Team, About R, R Homepage, The R Journal, Software, R Sources, R Binaries, Packages, Task Views, Other, Documentation, Manuals, FAQs, Contributed, Donations, and Donate. The main content area is titled "R-4.3.2 for Windows". A blue box highlights the "Download R-4.3.2 for Windows (79 megabytes, 64 bit)" link. Below this are links for "README on the Windows binary distribution" and "New features in this version". The text explains that the build requires UCRT, which is part of Windows 10 and Windows Server 2016, and provides a link to install it manually from here. It also mentions that the package can be compared to the master server using the md5sum of the .exe file. A "Frequently asked questions" section follows, with links for "Does R run under my version of Windows?" and "How do I update packages in my previous version of R?". A note directs users to the R FAQ for general information and the R Windows FAQ for Windows-specific information. An "Other builds" section lists links for "r-patched snapshot build", "r-devel snapshot build", and "Previous releases". A note to webmasters provides a stable link to the current Windows binary release. The page footer shows the URL "https://cran.rstudio.com/bin/windows/base/R-4.3.2-win.exe" and the text "Last change: 2023-10-31".

ice
nd
iti
an
itu
og

CRAN
Mirrors
What's new?
Search
CRAN Team
About R
R Homepage
The R Journal
Software
R Sources
R Binaries
Packages
Task Views
Other
Documentation
Manuals
FAQs
Contributed
Donations
Donate

R-4.3.2 for Windows

Download R-4.3.2 for Windows (79 megabytes, 64 bit)
[README on the Windows binary distribution](#)
[New features in this version](#)

This build requires UCRT, which is part of Windows since Windows 10 and Windows Server 2016. On older systems, UCRT has to be installed manually from [here](#).

If you want to double-check that the package you have downloaded matches the package distributed by CRAN, you can compare the [md5sum](#) of the .exe to the [fingerprint](#) on the master server.

Frequently asked questions

- [Does R run under my version of Windows?](#)
- [How do I update packages in my previous version of R?](#)

Please see the [R FAQ](#) for general information about R and the [R Windows FAQ](#) for Windows-specific information.

Other builds

- Patches to this release are incorporated in the [r-patched snapshot build](#).
- A build of the development version (which will eventually become the next major release of R) is available in the [r-devel snapshot build](#).
- [Previous releases](#)

Note to webmasters: A stable link which will redirect to the current Windows binary release is
[<CRAN MIRROR>/bin/windows/base/release.html](#)

Last change: 2023-10-31

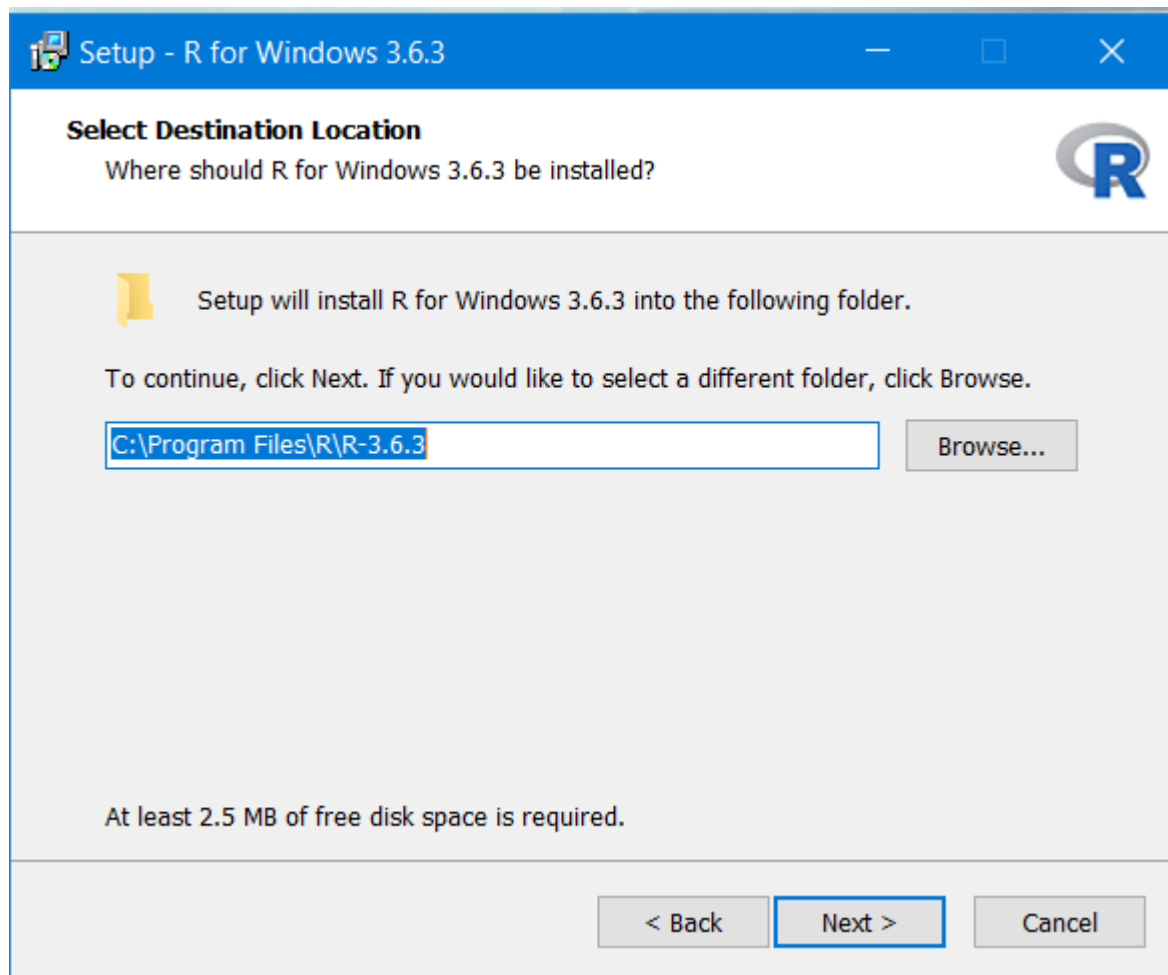
<https://cran.rstudio.com/bin/windows/base/R-4.3.2-win.exe>

Steps to Install R and R Studio

now click on the link show above in image so R base start downloading and after again go to main page and download and click on Install RStudio.

Steps to Install R and R Studio

Step 1: After downloading R for the Windows platform, install it by double-clicking it.



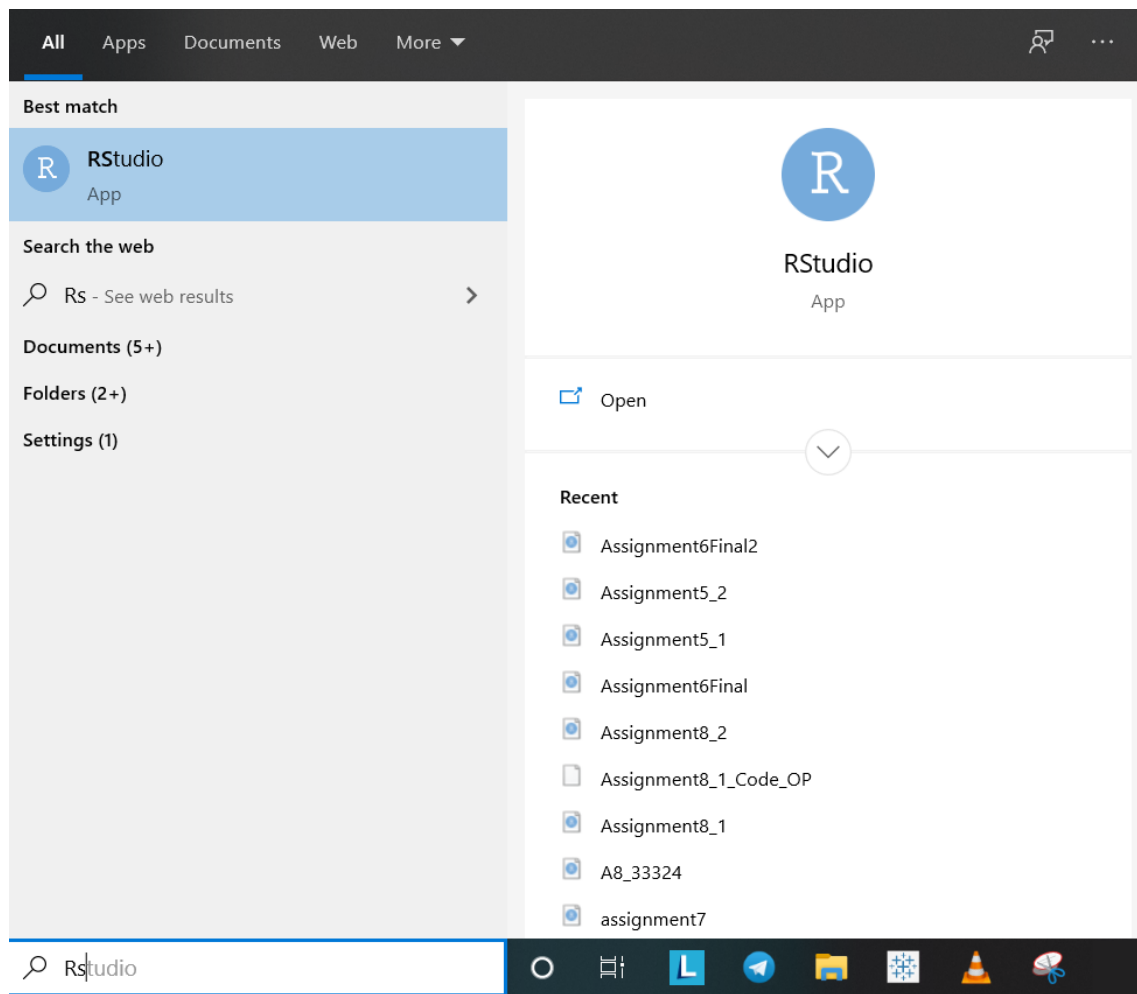
Step 2: Download R Studio from their official page. Note: It is free of cost (under AGPL licensing).

Step 3: After downloading, you will get a file named “RStudio-1.x.xxxx.exe” in your Downloads folder.

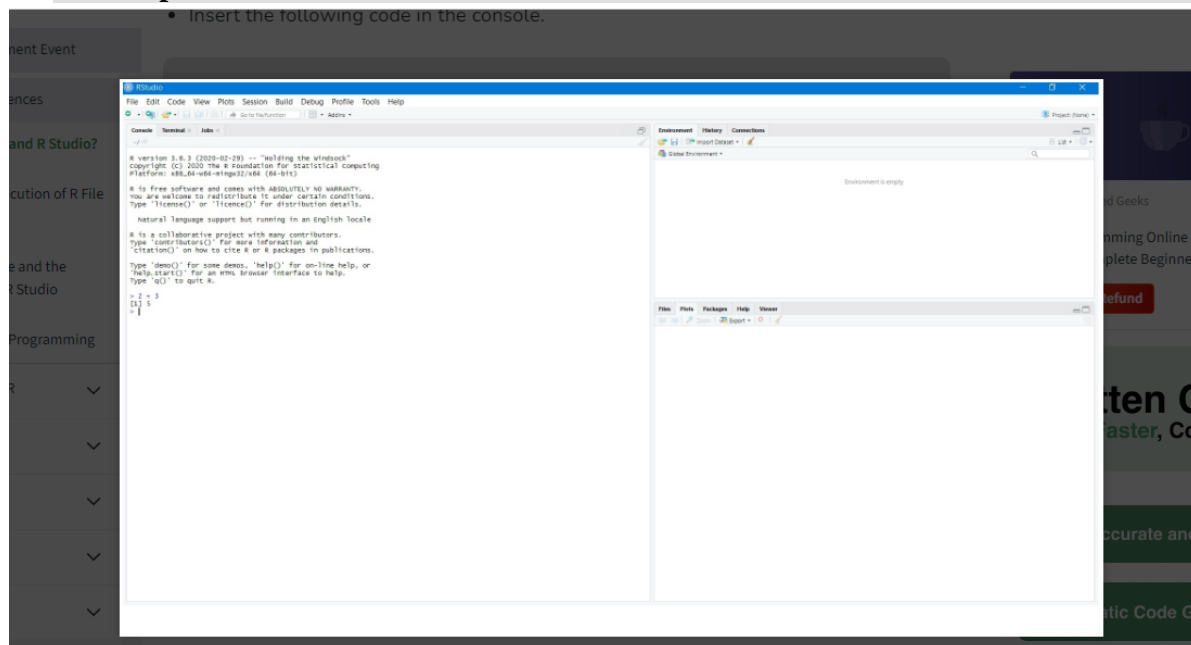
Step 4: Double-click the installer, and install the software.

Step 5: Test the R Studio installation

- Search for RStudio in the Window search bar on Taskbar.



- Start the application.
- Insert the following code in the console.
 - **Input** : `print("Hello world!")`
 - **Output** : `[1] "Hello world!"`



Step 6: Your installation is successful.

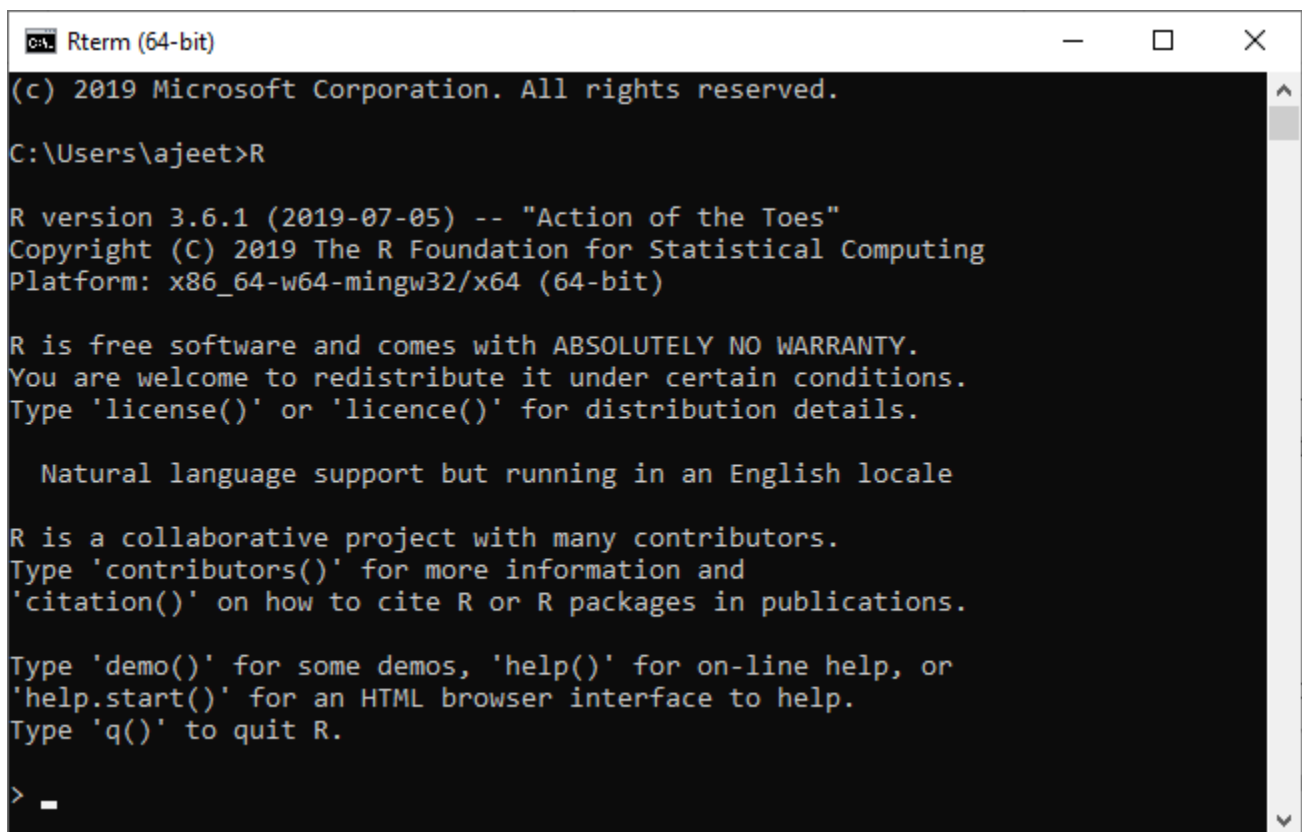
Basic Syntax in R Programming

Syntax of R Programming

R Programming is a very popular programming language which is broadly used in data analysis. The way in which we define its code is quite simple. The "Hello World!" is the basic program for all the languages, and now we will understand the syntax of R programming with "Hello world" program. We can write our code either in command prompt, or we can use an R script file.

R Command Prompt

It is required that we have already installed the R environment set up in our system to work on the R command prompt. After the installation of R environment setup, we can easily start R command prompt by typing R in our Windows command prompt. When we press enter after typing R, it will launch interpreter, and we will get a prompt on which we can code our program.

A screenshot of a Windows command prompt window titled "Rterm (64-bit)". The window has a black background with white text. The text shows the R startup sequence: a copyright notice for Microsoft, the command "C:\Users\ajeet>R", the R version "3.6.1 (2019-07-05)", copyright for The R Foundation, the platform "x86_64-w64-mingw32/x64 (64-bit)", a disclaimer about warranty, information about natural language support, contributors, and help options. The prompt "> _" is visible at the bottom.

```
Rterm (64-bit)
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\ajeet>R

R version 3.6.1 (2019-07-05) -- "Action of the Toes"
Copyright (C) 2019 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

  Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> _
```

"Hello, World!" Program

The code of "Hello World!" in R programming can be written as:

```
> string <- "Hello World!"  
> print(string)  
[1] "Hello World!"  
>
```

In the above code, the first statement defines a **string variable** string, where we assign a string "Hello World!". The next statement print() is used to print the value which is stored in the variable string.

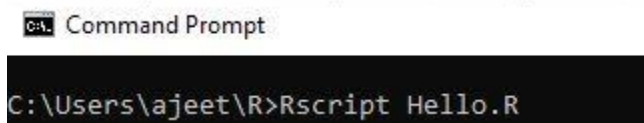
R Script File

The R script file is another way on which we can write our programs, and then we execute those scripts at our command prompt with the help of R interpreter known as **Rscript**. We make a text file and write the following code. We will save this file with .R extension as:

Demo.R

1. string <-"Hello World!"
2. print(string)

To execute this file in Windows and other operating systems, the process will remain the same as mentioned below.



```
C:\Users\ajeet\R>Rscript Hello.R
```

When we press enter it will give us the following output:



```
[1] "Hello World!"
```

Comments

In R programming, comments are the programmer readable explanation in the source code of an R program. The purpose of adding these comments is to make the source code easier to understand. These comments are generally ignored by compilers and interpreters.

Variables in R Programming

Variables are used to store the information to be manipulated and referenced in the R program. The R variable can store an atomic vector, a group of atomic vectors, or a combination of many R objects.

Language like C++ is statically typed, but R is a dynamically typed, means it check the type of data type when the statement is run. A valid variable name contains letter, numbers, dot and underlines characters. A variable name should start with a letter or the dot not followed by a number.

Name of variable	Validity	Reason for valid and invalid
_var_name	Invalid	Variable name can't start with an underscore(_).
var_name, var.name	Valid	Variable can start with a dot, but dot should not be followed by a number. In this case, the variable will be invalid.
var_name%	Invalid	In R, we can't use any special character in the variable name except dot and underscore.
2var_name	Invalid	Variable name cant starts with a numeric digit.
.2var_name	Invalid	A variable name cannot start with a dot which is followed by a digit.
var_name2	Valid	The variable contains letter, number and underscore and starts with a letter.

Assignment of variable

In R programming, there are three operators which we can use to assign the values to the variable. We can use leftward, rightward, and equal_to operator for this purpose.

There are two functions which are used to print the value of the variable i.e., print() and cat(). The cat() function combines multiples values into a continuous print output.

Assignment using equal operator.

variable.1 = 124

Assignment using leftward operator.

```
variable.2 <- "Learn R Programming"
```

Assignment using rightward operator.

```
133L -> variable.3
```

```
print(variable.1)
```

```
cat ("variable.1 is ", variable.1 , "\n")
```

```
cat ("variable.2 is ", variable.2 , "\n")
```

```
cat ("variable.3 is ", variable.3 , "\n")
```

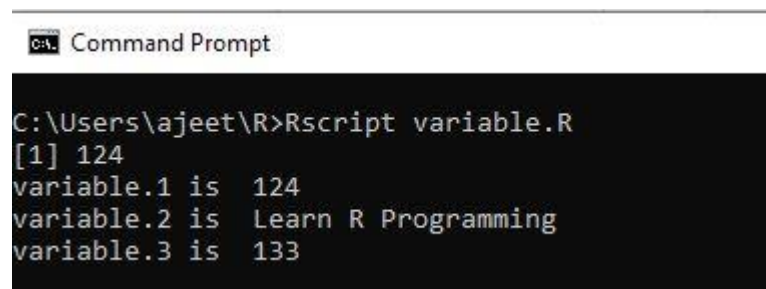
eg:

```
a.1=124
```

```
print(a.1)
```

```
cat ("a.1 is ", a.1 , "\n")
```

When we execute the above code in our R command prompt, it will give us the following output:



```
C:\> Command Prompt
C:\Users\ajeet\R>Rscript variable.R
[1] 124
variable.1 is 124
variable.2 is Learn R Programming
variable.3 is 133
```

Data types of variable

R programming is a dynamically typed language, which means that we can change the data type of the same variable again and again in our program. Because of its dynamic nature, a variable is not declared of any data type. It gets the data type from the R-object, which is to be assigned to the variable.

We can check the data type of the variable with the help of the `class()` function. Let's see an example:

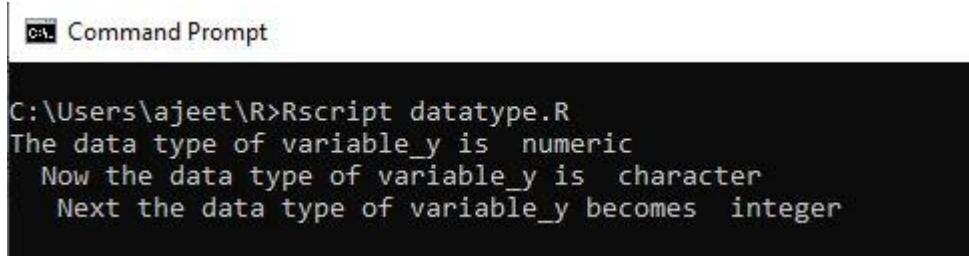
```
variable_y<- 124
```

```
cat("The data type of variable_y is ",class(variable_y), "\n")
```

```
variable_y<- "Learn R Programming"  
cat(" Now the data type of variable_y is ",class(variable_y),"\\n")
```

```
variable_y<- 133L  
cat(" Next the data type of variable_y becomes ",class(variable_y),"\\n")
```

When we execute the above code in our R command prompt, it will give us the following output:



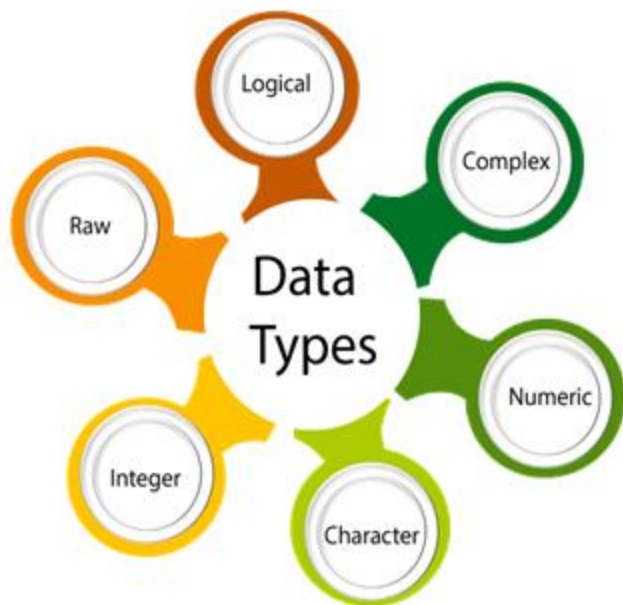
```
Command Prompt  
C:\\Users\\ajeet\\R>Rscript datatype.R  
The data type of variable_y is numeric  
Now the data type of variable_y is character  
Next the data type of variable_y becomes integer
```

Data Types in R Programming

In programming languages, we need to use various variables to store various information. Variables are the reserved memory location to store values. As we create a variable in our program, some space is reserved in memory.

In R, there are several data types such as integer, string, etc. The operating system allocates memory based on the data type of the variable and decides what can be stored in the reserved memory.

There are the following data types which are used in R programming:



Data type	Example	Description
Logical	True, False	It is a special data type for data with only two possible values which can be construed as true/false.
Numeric	12,32,112,5432	Decimal value is called numeric in R, and it is the default computational data type.
Integer	3L, 66L, 2346L	Here, L tells R to store the value as an integer,
Complex	Z=1+2i, t=7+3i	A complex value in R is defined as the pure imaginary value i.
Character	'a', "good", "TRUE", '35.4'	In R programming, a character is used to represent string values. We convert objects into character values with the help of as.character() function.
Raw		A raw data type is used to holds raw bytes.

Let's see an example for better understanding of data types:

#Logical Data type

```
variable_logical <- TRUE
```

```
cat(variable_logical, "\n")
```

```
cat("The data type of variable_logical is ",class(variable_logical),"\\n\\n")
```

#Numeric Data type

```
variable_numeric<- 3532
```

```
cat(variable_numeric,"\\n")
```

```
cat("The data type of variable_numeric is ",class(variable_numeric),"\\n\\n")
```

#Integer Data type

```
variable_integer<- 133L
```

```
cat(variable_integer,"\\n")
```

```
cat("The data type of variable_integer is ",class(variable_integer),"\\n\\n")
```

#Complex Data type

```
variable_complex<- 3+2i
```

```
cat(variable_complex,"\\n")
```

```
cat("The data type of variable_complex is ",class(variable_complex),"\\n\\n")
```

#Character Data type

```
variable_char<- "Learning r programming"
```

```
cat(variable_char,"\\n")
```

```
cat("The data type of variable_char is ",class(variable_char),"\\n\\n")
```

#Raw Data type

```
variable_raw<- charToRaw("Learning r programming")
```

```
cat(variable_raw,"\\n")
```

```
cat("The data type of variable_char is ",class(variable_raw),"\\n\\n")
```

When we execute the following program, it will give us the following output:

Command Prompt

```
C:\Users\ajeet\R>Rscript datatype.R
TRUE
The data type of variable_logical is logical

3532
The data type of variable_numeric is numeric

133
The data type of variable_integer is integer

3+2i
The data type of variable_complex is complex

Learning r programming
The data type of variable_char is character

4c 65 61 72 6e 69 6e 67 20 72 20 70 72 6f 67 72 61 6d 6d 69 6e 67
The data type of variable_char is raw
```

Generally, while doing programming in any programming language, you need to use various variables to store various information. Variables are nothing but reserved memory locations to store values. This means that, when you create a variable you reserve some space in memory. You may like to store information of various data types like character, wide character, integer, floating point, double floating point, Boolean etc. Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory. In contrast to other programming languages like C and java in R, the variables are not declared as some data type. The variables are assigned with R-Objects and the data type of the R-object becomes the data type of the variable. There are many types of R-objects. The frequently used ones are:

- Vectors
- Lists
- Matrices
- Arrays
- Factors
- Data Frames

Vectors

When you want to create vector with more than one element, you should use **c()** function which means to combine the elements into a vector.

```
# Create a vector.  
apple <- c('red','green',"yellow")  
print(apple)
```

R Programming

R

```
# Get the class of the vector.  
print(class(apple))
```

When we execute the above code, it produces the following result:

```
[1] "red"    "green"  "yellow"  
[1] "character"
```

Lists

A list is an R-object which can contain many different types of elements inside it like vectors, functions and even another list inside it.

```
# Create a list.  
list1 <- list(c(2,5,3),21.3,sin)  
  
# Print the list.  
print(list1)
```

When we execute the above code, it produces the following result:

```
[[1]]  
[1] 2 5 3  
  
[[2]]  
[1] 21.3  
  
[[3]]  
function (x) .Primitive("sin")
```

Matrices

A matrix is a two-dimensional rectangular data set. It can be created using a vector input to the matrix function.

```
# Create a matrix.  
M = matrix( c('a','a','b','c','b','a'), nrow=2,ncol=3,byrow = TRUE)  
print(M)
```

When we execute the above code, it produces the following result:

```
[,1] [,2] [,3]
```

9

```
[1,] "a"  "a"  "b"  
[2,] "c"  "b"  "a"
```

Arrays

While matrices are confined to two dimensions, arrays can be of any number of dimensions. The array function takes a dim attribute which creates the required number of dimension. In the below example we create an array with two elements which are 3x3 matrices each.

```
# Create an array.  
a <- array(c('green','yellow'),dim=c(3,3,2))  
print(a)
```

When we execute the above code, it produces the following result:

```
, , 1  
  
      [,1]      [,2]      [,3]  
[1,] "green"  "yellow" "green"  
[2,] "yellow" "green"  "yellow"  
[3,] "green"  "yellow" "green"  
  
, , 2  
  
      [,1]      [,2]      [,3]  
[1,] "yellow" "green"  "yellow"  
[2,] "green"  "yellow" "green"  
[3,] "yellow" "green"  "yellow"
```

Factors

Factors are the r-objects which are created using a vector. It stores the vector along with the distinct values of the elements in the vector as labels. The labels are always character irrespective of whether it is numeric or character or Boolean etc. in the input vector. They are useful in statistical modeling.

Factors are created using the **factor()** function. The **nlevels** functions gives the count of levels.

```
# Create a vector.  
apple_colors <- c('green','green','yellow','red','red','red','green')
```

10

```
# Create a factor object.  
factor_apple <- factor(apple_colors)  
  
# Print the factor.  
print(factor_apple)  
print(nlevels(factor_apple))
```

When we execute the above code, it produces the following result:

```
[1] green green yellow red    red    red    yellow green  
Levels: green red yellow  
# applying the nlevels function we can know the number of distinct values  
[1] 3
```

Data Frames

Data frames are tabular data objects. Unlike a matrix in data frame each column can contain different modes of data. The first column can be numeric while the second column can be character and third column can be logical. It is a list of vectors of equal length.

Data Frames are created using the **data.frame()** function.

```
# Create the data frame.
BMI <- data.frame(
  gender = c("Male", "Male", "Female"),
  height = c(152, 171.5, 165),
  weight = c(81, 93, 78),
  Age = c(42, 38, 26)
)
print(BMI)
```

When we execute the above code, it produces the following result:

	gender	height	weight	Age
1	Male	152.0	81	42
2	Male	171.5	93	38
3	Female	165.0	78	26

Importing data into R from different file formats (CSV, Excel, etc.).

Importing Data in R Script

•

We can read external datasets and operate with them in our R environment by importing data into an R script. R offers a number of functions for importing data from various file formats.

In this article, we are going to see how to **Import data in [R Programming Language](#)**.

Importing Data in R

First, let's consider a data set that we can use for the demonstration. For this demonstration, we will use two examples of a single dataset, one in .csv form and another .txt

A	B	C
6.475	6	62.1
10.125	18	74.7
9.55	16	69.7
11.125	14	71
4.8	5	56.9
6.225	11	58.7
4.95	8	63.3
7.325	11	70.4
8.875	15	70.5
6.8	11	59.2

Importing Data in R Script

Reading a Comma-Separated Value(CSV) File

Method 1: Using read.csv() Function Read CSV Files into R

The function has two parameters:

- **file.choose():** It opens a menu to choose a CSV file from the desktop.
- **header:** It is to indicate whether the first row of the dataset is a variable name or not. Apply T/True if the variable name is present else put F/False.

Example:

- R

```
# import and store the dataset in data1

data1 <- read.csv(file.choose(), header=T)


# display the data

data1
```

Output:

	A	B	C
1	6.475	6	62.1
2	10.125	18	74.7
3	9.550	16	69.7
4	11.125	14	71.0
5	4.800	5	56.9
6	6.225	11	58.7
7	4.950	8	63.3
8	7.325	11	70.4
9	8.875	15	70.5
10	6.800	11	59.2

Importing Data in R Script

Method 2: Using read.table() Function

This function specifies how the dataset is separated, in this case we take **sep=**”, “ as an argument.

Example:

- R

```
# import and store the dataset in data2

data2 <- read.table(file.choose(), header=T, sep=", ")

# display data

data2
```

Output:

	A	B	C
1	6.475	6	62.1
2	10.125	18	74.7
3	9.550	16	69.7
4	11.125	14	71.0
5	4.800	5	56.9
6	6.225	11	58.7
7	4.950	8	63.3
8	7.325	11	70.4
9	8.875	15	70.5
10	6.800	11	59.2

Importing Data in R Script

Reading a Tab-Delimited(txt) File in R Programming Language

Method 1: Using read.delim() Function

The function has two parameters:

- **file.choose():** It opens a menu to choose a csv file from the desktop.
- **header:** It is to indicate whether the first row of the dataset is a variable name or not. Apply T/True if the variable name is present else put F/False.

Example:

• R

```
# import and store the dataset in data3

data3 <- read.delim(file.choose(), header=T)


# display the data

data3
```

Output:

	A	B	C
1	6.475	6	62.1
2	10.125	18	74.7
3	9.550	16	69.7
4	11.125	14	71.0
5	4.800	5	56.9
6	6.225	11	58.7
7	4.950	8	63.3
8	7.325	11	70.4
9	8.875	15	70.5
10	6.800	11	59.2

Importing Data in R Script

Method 2: Using read.table() Function

This function specifies how the dataset is separated, in this case we take **sep="\t"** as the argument.

Example:

- R

```
# import and store the dataset in data4

data4 <- read.table(file.choose(), header=T, sep="\t")

# display the data

data4
```

Output:

	A	B	C
1	6.475	6	62.1
2	10.125	18	74.7
3	9.550	16	69.7
4	11.125	14	71.0
5	4.800	5	56.9
6	6.225	11	58.7
7	4.950	8	63.3
8	7.325	11	70.4
9	8.875	15	70.5
10	6.800	11	59.2

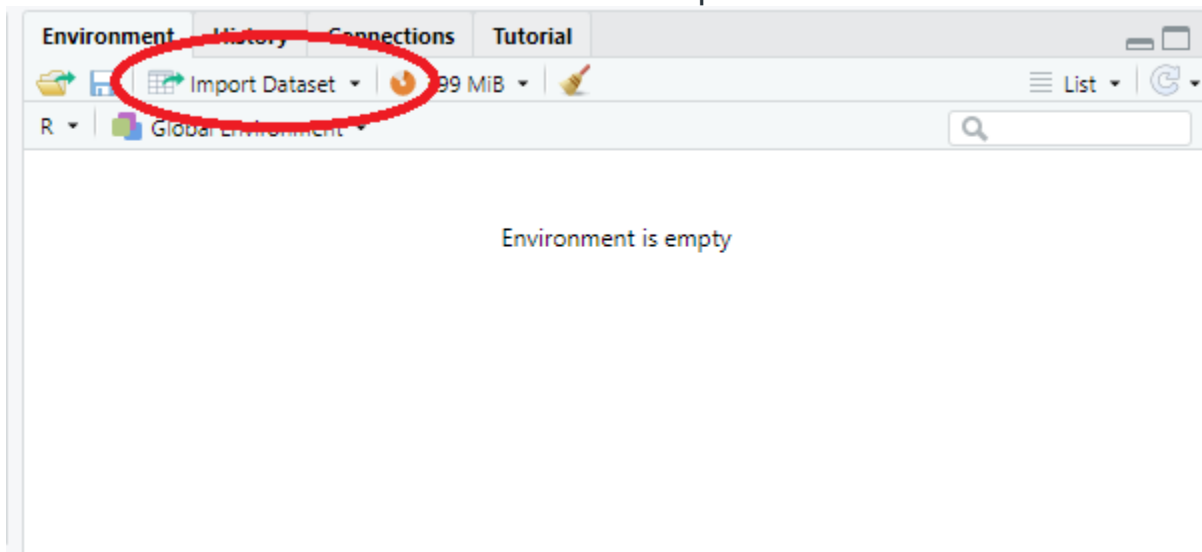
Importing Data in R Script

Using R-Studio

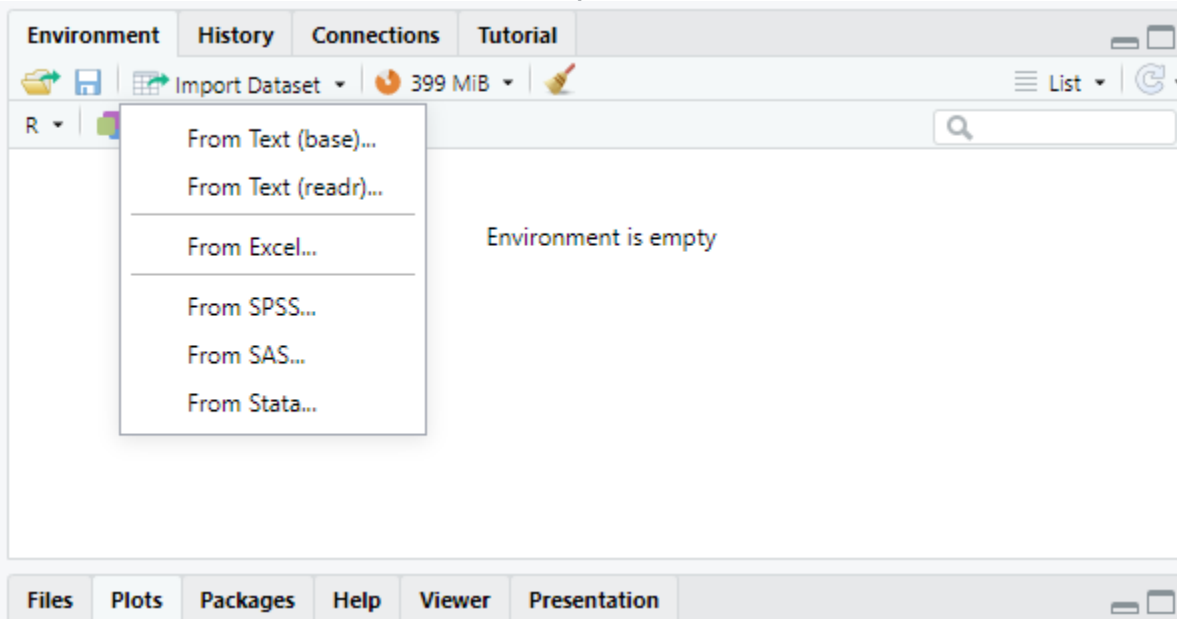
Here we are going to import data through R studio with the following steps.

Steps:

- From the Environment tab click on the Import Dataset Menu.



- Select the file extension from the option.



- In the third step, a pop-up box will appear, either enter the file name or browse the desktop.
- The selected file will be displayed on a new window with its dimensions.
- In order to see the output on the console, type the filename.

Read JSON Files Into R

In order to work with JSON files in [R](#), one needs to install the “rjson” package. The most common tasks done using JSON files under rjson packages are as follows:

- Install and load the rjson package in R console
- Create a JSON file
- Reading data from JSON file
- Write into JSON file
- Converting the JSON data into Dataframes
- Working with URLs

JSON file for demonstration:

```
{  
  "ID":["1","2","3","4","5"],  
  "Name":["Mithuna","Tanushree","Parnasha","Arjun","Pankaj"],  
  "Salary":["722.5","815.2","1611","2829","843.25"],
```

```
"StartDate":["6/17/2014","1/1/2012","11/15/2014","9/23/2013","5/21/2013"],
  "Dept":["IT","IT","HR","Operations","Finance"]
}
```

Code:

- R

```
# Read a JSON file

# Load the package required to read JSON files.

library("rjson")

# Give the input file name to the function.

result <- fromJSON(file = "E:\\example.json")

# Print the result.

print(result)
```

Output:

\$ID

```
[1] "1" "2" "3" "4" "5"
```

\$Name

```
[1] "Mithuna" "Tanushree" "Parnasha" "Arjun" "Pankaj"
```

\$Salary

```
[1] "722.5" "815.2" "1611" "2829" "843.25"
```

```
$StartDate
```

```
[1] "6/17/2014" "1/1/2012" "11/15/2014" "9/23/2013" "5/21/2013"
```

```
$Dept
```

```
[1] "IT" "IT" "HR" "Operations" "Finance"
```

Working with CSV files in R Programming

- In this article, we will discuss how to work with CSV files in R Programming Language.

R CSV Files

R CSV Files are text files wherein the values of each row are separated by a delimiter, as in a comma or a tab. In this article, we will use the following sample CSV file.

Getting and Setting the Working Directory with R CSV Files

- R

```
# Get the current working directory.

print(getwd())

# Set current working directory.

setwd("/web/com")

# Get and print current working directory.

print(getwd())
```

Output:

```
[1] "C:/Users/GFG19565/Documents"
```

```
[1] "C:/Users/GFG19565/Documents"
```

With the help of `getwd()` function we can get the current working directory and with the help of `setwd()` function we can also set a new working directory.

Input as R CSV Files

id	name	department	salary	projects
1,	A,	IT,	60754,	4
2,	B,	Tech,	59640,	2
3,	C,	Marketing,	69040,	8
4,	D,	Marketing,	65043,	5
5,	E,	Tech,	59943,	2
6,	F,	IT,	65000,	5
7,	G,	HR,	69000,	7

We can save this file in notepad and give name **sample.csv** so we can upload this in R Programming Language.

Reading a R CSV Files

The contents of a CSV file can be read as a data frame in R using the `read.csv(...)` function. The CSV file to be read should be either present in the current working directory or the directory should be set accordingly using the `setwd(...)` command in R. The CSV file can also be read from a URL using `read.csv()` function.

- R

```
csv_data <- read.csv(file = 'C:\\Users\\GFG19565\\Downloads\\sample.csv')

print(csv_data)

# print number of columns

print (ncol(csv_data))

# print number of rows

print(nrow(csv_data))
```

Output:

Output:

	id,	name,	department,	salary,	projects
1	1	A	HR	60754	14
2	2	B	Tech	59640	3
3	3	C	Marketing	69040	8
4	4	D	HR	65043	5
5	5	E	Tech	59943	2
6	6	F	IT	65000	5
7	7	G	HR	69000	7

```
[1] 4
[1] 7
```

We can upload the R Csv Files by passing its directory the header is by default set to a TRUE value in the function. The head is not included in the count of rows, therefore this CSV has 7 rows and 4 columns.

Querying with R CSV Files

SQL queries can be performed on the CSV content, and the corresponding result can be retrieved using the `subset(csv_data,)` function in R. Multiple queries can be applied in the function at a time where each query is separated using a logical operator. The result is stored as a data frame in R.

- R

```
min_pro <- min(csv_data$projects)

print (min_pro)
```

Output:

```
2
```

Aggregator functions (min, max, count etc.) can be applied on the CSV data. Here the **min()** function is applied on projects column using \$ symbol. The minimum number of projects which is 2 is returned.

- R

```
# Selecting 'name' and 'salary' columns for employees with salary greater than
60000

result <- csv_data[csv_data$salary > 60000, c("name", "salary")]
```

```
# Print the result
```

```
print(result)
```

```
  name salary
1    A  60754
3    C  69040
4    D  65043
6    F  65000
7    G  69000
```

The subset of the data that is created is stored as a data frame satisfying the conditions specified as the arguments of the function. Selecting 'name' and 'salary' columns for employees with salary greater than 60000.

Writing into a R CSV Files

The contents of the data frame can be written into a CSV file. The CSV file is stored in the current working directory with the name specified in the function `write.csv(data frame, output CSV name)` in R.

- R

```
# Calculating the average salary for each department
```

```
result <- tapply(csv_data$salary, csv_data$department, mean)
```

```
# Print the result
```

```
print(result)
```

Output:

HR	IT	Marketing	Tech
69000.0	62877.0	67041.5	59791.5

In this we will Calculating the average salary for each department.

Working with Excel Files in R Programming

Last Updated : 12 Jan, 2022

Excel files are of extension .xls, .xlsx and .csv(comma-separated values). To start working with excel files in [R Programming Language](#), we need to first

import excel files in RStudio or any other R supporting IDE(Integrated development environment).

Reading Excel Files in R Programming Language

First, install **readxl** package in R to load excel files. Various methods including their subparts are demonstrated further.

Sample_data1.xlsx:

PassengerId	Survived	Pclass	Name	Sex	Age
1	0	3	Braund, Mr. Owen Harris	male	22
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38
3	1	3	Heikkinen, Miss. Laina	female	26
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35
5	0	3	Allen, Mr. William Henry	male	35
6	0	3	Moran, Mr. James	male	46
7	0	1	McCarthy, Mr. Timothy J	male	54
8	0	3	Palsson, Master. Gosta Leonard	male	2
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27
10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14
11	1	3	Sandstrom, Miss. Marguerite Rut	female	4
12	1	1	Bonnell, Miss. Elizabeth	female	58
13	0	3	Saunderscock, Mr. William Henry	male	20
14	0	3	Andersson, Mr. Anders Johan	male	39
15	0	3	Vestrom, Miss. Hulda Amanda Adolfina	female	14
16	1	2	Hewlett, Mrs. (Mary D Kingcome)	female	55
17	0	3	Rice, Master. Eugene	male	2
18	1	2	Williams, Mr. Charles Eugene	male	3
19	0	3	Vander Planke, Mrs. Julius (Emelia Maria Vandemoortele)	female	31
20	1	3	Masselmani, Mrs. Fatima	female	33
21	0	2	Fynney, Mr. Joseph J	male	35
22	1	2	Beesley, Mr. Lawrence	male	34
23	1	3	McGowan, Miss. Anna "Annie"	female	15
24	1	1	Sloper, Mr. William Thompson	male	28
25	0	3	Palsson, Miss. Torborg Danira	female	8
26	1	3	Asplund, Mrs. Carl Oscar (Selma Augusta Emilia Johansson)	female	38
27	0	3	Emir, Mr. Farred Chehab	male	33

Sample_data2.xlsx:

SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	A/5 21171	7.25	C65	S
1	0	PC 17599	71.2833	C85	C
0	0	3101282	7.925	C45	S
1	0	113803	53.1	C123	S
0	0	373450	8.05	C34	S
0	0	330877	8.4583	C86	Q
0	0	17463	51.8625	E46	S
3	1	349909	21.075	D73	S
0	2	347742	11.1333	G44	S
1	0	237736	30.0708	H5	C
1	1	PP 9549	16.7	G6	S
0	0	113783	26.55	C103	S
0	0	A/5. 2151	8.05	C45	S
1	5	347082	31.275	C56	S
0	0	350406	7.8542	V47	S
0	0	248706	16	G43	S
4	1	382652	29.125	F23	Q
0	0	244373	13	H34	S
1	0	345763	18	F22	S
0	0	2649	7.225	G233	C
0	0	239865	26	F23	S
0	0	248698	13	D56	S
0	0	330923	8.0292	G12	Q
0	0	113788	35.5	A6	S
3	1	349909	21.075	S34	S
1	5	347077	31.3875	S35	S
0	0	2631	7.225	E23	C

Reading Files:

The two excel files Sample_data1.xlsx and Sample_data2.xlsx and read from the working directory.

- R

```
# Working with Excel Files

# Installing required package

install.packages("readxl")


# Loading the package

library(readxl)
```

```
# Importing excel file

Data1 <- read_excel("Sample_data1.xlsx")

Data2 <- read_excel("Sample_data2.xlsx")

# Printing the data

head(Data1)

head(Data2)
```

```
> head(Data1)
# A tibble: 6 x 6
  PassengerId Survived Pclass Name Sex
  <dbl>     <dbl>   <dbl> <chr> <chr>
1         1         0       3 Braund, Mr. Owen Harris male
2         2         1       1 Cumings, Mrs. John Bradley (Florence Briggs Thayer) female
3         3         1       3 Heikkinen, Miss. Laina female
4         4         1       1 Futrelle, Mrs. Jacques Heath (Lily May Peel) female
5         5         0       3 Allen, Mr. William Henry male
6         6         0       3 Moran, Mr. James male

> head(Data2)
# A tibble: 6 x 6
  SibSp Parch Ticket Fare Cabin Embarked
  <dbl> <dbl> <chr>     <dbl> <chr> <chr>
1     1     0 A/5 21171    7.25 C65   S
2     1     0 PC 17599   71.3 C85   C
3     0     0 STON/O2. 3101282 7.92 C45   S
4     1     0 113803    53.1 C123  S
5     0     0 373450     8.05 C34   S
6     0     0 330877     8.46 C86   Q
```

The excel files are loaded into variables Data_1 and Data_2 as a **dataframes** and then variable Data_1 and Data_2 is called that prints the dataset.

Modifying Files

The Sample_data1.xlsx file and Sample_file2.xlsx are modified.

- R

```
# Modifying the files
```

```
Data1$Pclass <- 0
```

```
Data2$Embarked <- "S"
```

```
# Printing the data
```

```
head(Data1)
```

```
head(Data2)
```

```
> head(Data1)
# A tibble: 6 x 6
  PassengerId Survived Pclass Name                               Sex
  <dbl>      <dbl>   <dbl> <chr>                               <chr>
1         1         0       0 Braund, Mr. Owen Harris          male
2         2         1       0 Cumings, Mrs. John Bradley (Florence Briggs Thayer) female
3         3         1       0 Heikkinen, Miss. Laina          female
4         4         1       0 Futrelle, Mrs. Jacques Heath (Lily May Peel)    female
5         5         0       0 Allen, Mr. William Henry       male
6         6         0       0 Moran, Mr. James              male

> head(Data2)
# A tibble: 6 x 6
  SibSp Parch Ticket            Fare Cabin Embarked
  <dbl> <dbl> <chr>            <dbl> <chr> <chr>
1     1     0 A/5 21171         7.25 C65   S
2     1     0 PC 17599        71.3 C85   S
3     0     0 STON/O2. 3101282  7.92 C45   S
4     1     0 113803         53.1 C123  S
5     0     0 373450         8.05 C34   S
6     0     0 330877         8.46 C86   S
```

The value of the P-class attribute or variable of Data1 data is modified to 0.
The value of Embarked attribute or variable of Data2 is modified to S.

Deleting Content from files

The variable or attribute is deleted from Data1 and Data2 datasets containing Sample_data1.xlsx and Sample_data2.xlsx files.

- R

```
# Deleting from files
```

```
Data1 <- Data1[-2]
```

```
Data2 <- Data2[-3]
```

```
# Printing the data
```

```
Data1
```

```
Data2
```

```
> head(Data1)
# A tibble: 6 x 5
  PassengerId Pclass Name                      Sex    Age
  <dbl>    <dbl> <chr>                      <chr> <dbl>
1         1      0 Braund, Mr. Owen Harris    male   22
2         2      0 Cumings, Mrs. John Bradley (Florence Briggs Thayer) female  38
3         3      0 Heikkinen, Miss. Laina     female  26
4         4      0 Futrelle, Mrs. Jacques Heath (Lily May Peel)    female  35
5         5      0 Allen, Mr. William Henry   male   35
6         6      0 Moran, Mr. James          male   46

> head(Data2)
# A tibble: 6 x 5
  SibSp Parch  Fare Cabin Embarked
  <dbl> <dbl> <dbl> <chr> <chr>
1     1     0  7.25 C65   S
2     1     0 71.3  C85   S
3     0     0  7.92 C45   S
4     1     0 53.1  C123  S
5     0     0  8.05 C34   S
6     0     0  8.46 C86   S
```

The – sign is used to delete columns or attributes from the dataset. Column 2 is deleted from the Data1 dataset and Column 3 is deleted from the Data2 dataset.

Merging Files

The two excel datasets Data1 and Data2 are merged using **merge()** function which is in base package and comes pre-installed in R.

- R

```
# Merging Files
```

```
Data3 <- merge(Data1, Data2, all.x = TRUE, all.y = TRUE)
```

```
# Displaying the data
```

```
head(Data3)
```

```
> head(Data3)
  PassengerId  Survived  Age  SibSp  Parch Name           Sex   Age  SibSp
1          1         0    22     1     0 Braund, Mr. Owen Harris male   22     1
2          2         0   38     1     0 Cumings, Mrs. John Bradley (Florence Briggs Thayer) female  38     1
3          3         0   26     1     0 Heikkinen, Miss. Laina female   26     1
4          4         0   35     1     0 Futrelle, Mrs. Jacques Heath (Lily May Peel) female   35     1
5          5         0   35     1     0 Allen, Mr. William Henry male    35     1
6          6         0   46     1     0 Moran, Mr. James male    46     1
  Parch  Fare  Cabin Embarked
1      0  7.25   C65        S
2      0  7.25   C65        S
3      0  7.25   C65        S
4      0  7.25   C65        S
5      0  7.25   C65        S
6      0  7.25   C65        S
```

Data1 and Data2 are merged with each other and the resultant file is stored in the Data3 variable.

Creating new columns

New columns or features can be easily created in Data1 and Data2 datasets.

- R

```
# Creating feature in Data1 dataset
```

```
Data1$Num <- 0
```

```
# Creating feature in Data2 dataset
```

```
Data2$Code <- "Mission"
```

```
# Printing the data
```

```
head(Data1)
```

```
head(Data2)
```

```
> head(Data1)
# A tibble: 6 x 6
  PassengerId Pclass Name Sex Age
  <dbl> <dbl> <chr> <chr> <dbl>
1 1 1 0 Braund, Mr. Owen Harris male 22
2 2 2 0 Cumings, Mrs. John Bradley (Florence Briggs Thayer) female 38
3 3 3 0 Heikkinen, Miss. Laina female 26
4 4 4 0 Futrelle, Mrs. Jacques Heath (Lily May Peel) female 35
5 5 5 0 Allen, Mr. William Henry male 35
6 6 6 0 Moran, Mr. James male 46

> head(Data2)
# A tibble: 6 x 6
  SibSp Parch Fare Cabin Embarked Code
  <dbl> <dbl> <dbl> <chr> <chr> <chr>
1 1 1 0 7.25 C65 S Mission
2 1 0 71.3 C85 S Mission
3 0 0 7.92 C45 S Mission
4 1 0 53.1 C123 S Mission
5 0 0 8.05 C34 S Mission
6 0 0 8.46 C86 S Mission
```

Num is a new feature that is created with 0 default value in Data1 dataset. **Code** is a new feature that is created with the mission as a default string in Data2 dataset.

Writing Files

After performing all operations, Data1 and Data2 are written into new files using **write.xlsx()** function built in writexl package.

- R

```
# Installing the package
```

```
install.packages("writexl")
```

```
# Loading package
```

```
library(writexl)

# Writing Data1

write_xlsx(Data1, "New_Data1.xlsx")

# Writing Data2

write_xlsx(Data2, "New_Data2.xlsx")
```

PassengerId	Pclass	Name	Sex	Age	Num
1	0	Braund, Mr. Owen Harris	male	22	0
2	0	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	0
3	0	Heikkinen, Miss. Laina	female	26	0
4	0	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	0
5	0	Allen, Mr. William Henry	male	35	0
6	0	Moran, Mr. James	male	46	0
7	0	McCarthy, Mr. Timothy J	male	54	0
8	0	Palsson, Master. Gosta Leonard	male	2	0
9	0	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0
10	0	Nasser, Mrs. Nicholas (Adele Achem)	female	14	0
11	0	Sandstrom, Miss. Marguerite Rut	female	4	0
12	0	Bonnell, Miss. Elizabeth	female	58	0
13	0	Saunderscock, Mr. William Henry	male	20	0
14	0	Andersson, Mr. Anders Johan	male	39	0
15	0	Vestrom, Miss. Hulda Amanda Adolfina	female	14	0
16	0	Hewlett, Mrs. (Mary D Kingcome)	female	55	0
17	0	Rice, Master. Eugene	male	2	0
18	0	Williams, Mr. Charles Eugene	male	3	0
19	0	Vander Planke, Mrs. Julius (Emelia Maria Vandemoortele)	female	31	0
20	0	Masselmani, Mrs. Fatima	female	33	0
21	0	Fynney, Mr. Joseph J	male	35	0
22	0	Beesley, Mr. Lawrence	male	34	0
23	0	McGowan, Miss. Anna "Annie"	female	15	0
24	0	Sloper, Mr. William Thompson	male	28	0
25	0	Palsson, Miss. Torborg Danira	female	8	0
26	0	Asplund, Mrs. Carl Oscar (Selma Augusta Emilia Johansson)	female	38	0
27	0	Emir, Mr. Farred Chehab	male	33	0

SibSp	Parch	Fare	Cabin	Embarked	Code
1	0	7.25	C65	S	Mission
1	0	71.2833	C85	S	Mission
0	0	7.925	C45	S	Mission
1	0	53.1	C123	S	Mission
0	0	8.05	C34	S	Mission
0	0	8.4583	C86	S	Mission
0	0	51.8625	E46	S	Mission
3	1	21.075	D73	S	Mission
0	2	11.1333	G44	S	Mission
1	0	30.0708	H5	S	Mission
1	1	16.7	G6	S	Mission
0	0	26.55	C103	S	Mission
0	0	8.05	C45	S	Mission
1	5	31.275	C56	S	Mission
0	0	7.8542	V47	S	Mission
0	0	16	G43	S	Mission
4	1	29.125	F23	S	Mission
0	0	13	H34	S	Mission
1	0	18	F22	S	Mission
0	0	7.225	G233	S	Mission
0	0	26	F23	S	Mission
0	0	13	D56	S	Mission
0	0	8.0292	G12	S	Mission
0	0	35.5	A6	S	Mission
3	1	21.075	S34	S	Mission
1	5	31.3875	S35	S	Mission
0	0	7.225	E23	S	Mission

The Data1 dataset is written **New_Data1.xlsx** file and Data2 dataset is written in **New_Data2.xlsx** file. Both the files are saved in the present working directory.